TrustHome PH

QA & Testing Strategy

How TrustHome is tested, how evidence is labelled, how to use the Use Cases & Test Cases workbook, and the scenarios that must pass before any demo or release.

Document	TH-08 · QA & Testing Strategy
Version / date	Rev. 2.0 · 4 October 2026
Factual baseline	TrustHome_Current_Progress_Report_2026-10-04.md (PR #42 `fixes` working copy, based on `features/v1-3-navigation-settings`; excludes `dev`)
Replaces	New (was spread across plans)
Audience	Team Idle developers, Scrum Master, QA, panel/adviser reviewers, and AI coding assistants used by the team

Evidence label	Meaning (never use these interchangeably)
[Implemented in source]	Code exists in the repository working copy. Not proof that it runs in production.
[Unit/emulator tested]	Covered by automated unit tests or the Firestore emulator suite.
[Reported working]	A team member observed it working in the browser against the test project.
[Deployed/live verified]	Deployed to trusthome-ph and confirmed working there.
[Known gap]	Does not yet meet the product claim. Do not present it as working.
[Roadmap] / **[Proposed]**	Planned future work / a recommendation in this document, not yet built.

Contents

1. Quality Goals . . . 3
2. Evidence Levels . . . 3
3. Test Levels . . . 3
4. The Use Cases & Test Cases Workbook . . . 4
4.1 Executing a test . . . 4
4.2 Status values . . . 4
5. Critical Scenarios (run before every demo) . . . 5
6. Security Test Matrix (rules) . . . 5
7. Defect and Gap Management . . . 6
8. Non-functional Checks . . . 6

1. Quality Goals

- **Correctness of the core promises**: verified-only publication, no private data leakage, valid booking/dispute state transitions, correct trust/fairness math.
- **Security at the data layer**: every forbidden action is denied by Firestore rules even when the UI is bypassed.
- **Honest evidence**: every claim in a demo or report can be traced to a test result with an evidence label.
- **Usable and accessible** on a 390 px phone and a 1280 px admin laptop, in light and dark themes.

2. Evidence Levels

Label	What counts as proof	What does NOT count
[Implemented in source]	Code reviewed and merged in the branch being described	An open PR; code on another branch
[Unit/emulator tested]	A named automated test that passed on a recorded run (date, commit)	A test that exists but was not run after the last change
[Reported working]	A tester's browser run with test account, date, steps and screenshot recorded in the workbook	"It worked for me" in chat
[Deployed/live verified]	Executed on the deployed `trusthome-ph` site/rules from a known commit	Local dev server against the shared DB without deploying
[Known gap]	Analysis or test shows the claim is not met	—

> **RULE · Status discipline**
> Never upgrade a label in a report without new evidence. "Passed (Dev)" in the workbook means executed in development/emulator; it becomes plain "Passed" only when re-run on the deployed build.

3. Test Levels

Level	Tooling	Scope	Latest evidence (baseline)
Unit (JS)	Node test runner (`npm test`)	Pure logic in `src/lib/`: validation, overlap, transitions, trust/fairness math, admin-guard policy	48 passed (previous fresh run)
Static	ESLint (`npm run lint`)	Whole frontend	Passed
Build	`npm run build`	Production bundle	Passed
Rules	Firestore emulator + `scripts/firestore-rules-smoke.mjs`	Allow/deny matrix per role and transition	9 passed (incl. owner approval, owner-scoped booking query)
Unit (Python)	`unittest`	Scraper parsing/validation, upsert/delist, trust scores, backup/restore, migration	28 passed
Integration	Emulator or test project with seed data	Jobs (expiry, trust recompute), notifications, batches/transactions	Partial
End-to-end / UAT	Manual scripted walkthrough (TC-E2E-001), later Playwright	Owner → admin → renter → dispute on the deployed build	Not yet run on deployed fixes branch
Non-functional	Lighthouse/axe, keyboard, screen reader, throttled network, viewport checks	Accessibility, performance, resilience	Not yet run

BEST PRACTICE · **Test pyramid for this project**

- Most tests at the unit level (fast, deterministic) for every business rule in TH-03.
- Every rule in `firestore.rules` gets at least one allow and one deny emulator test — this is the security proof.
- A small number of scripted end-to-end runs on the deployed build before each demo.
- **[Proposed]** Automate TC-E2E-001 with Playwright (Chromium is available in CI) against the emulator or a preview channel.

4. The Use Cases & Test Cases Workbook

Sheet	Purpose
Read Me	How to use the workbook, what changed in the 2026-10-04 revision
Use Cases	UC-01…UC-47: actor, preconditions, main/alternate flows, postconditions, source document, version, **evidence (Oct 4)**
Test Cases	All test cases with traceability (UC, FR), type, priority, steps, data, expected result, origin, status and execution columns
Summary	Live COUNTIFS by module, status and origin; pass rate
Requirements	Revised and new FRs (FR-002 … FR-028) with acceptance tests
Known Gaps	GAP-01 … register of release risks with severity, owner, linked tests and exit criteria

4.1 Executing a test

1. Pick rows by module and priority (Critical/High first). Check preconditions and test data.
2. Run on the environment stated in the Remarks (emulator, dev, or deployed). Use test accounts only.
3. Fill the yellow columns: **Status** (dropdown), **Actual Result** (what you observed, not “ok”), **Tester**, **Date** (YYYY-MM-DD).
4. For Failed: add a screenshot to the team drive, create/link a defect or GAP ID in Remarks.
5. Do not edit Expected Result to match what happened. If the requirement changed, update the FR and mark the row Origin “Updated”.

4.2 Status values

Status	Meaning
Passed	Passed on the deployed build
Passed (Dev)	Passed in development, emulator or unit test — not yet on deployed build
Failed	Executed and did not meet the expected result
Blocked	Cannot be executed because of an environment dependency (state which one)
Not Run	Ready but not executed (including cases expected to fail because of a known gap — say so in Remarks)
Retired	Feature removed; kept for traceability

5. Critical Scenarios (run before every demo)

Area	Scenario	Test IDs
Access	Regular, suspended and profile-less users cannot render or query admin data	TC-ADM-003…008
Privacy	No exact address or document URL in public listing data; listingPrivate owner/admin only	TC-VER-006, TC-MAP-004, TC-MAP-007, TC-MYL-005
Intake	All validation rules incl. 4 photos, both documents, showing window, 400-word cap	TC-INTAKE-001…015
Review	Approve / request changes / reject with notifications; document preview/download	TC-VER-001…009
Booking	Request, overlap rejection, approve, decline, sale not bookable, concurrency	TC-BOOK-001…023
Disputes	Submit → Disputed; Founded suspends + counts; Dismissed restores	TC-DISP-001…009
Trust	Formula, recency, fairness thresholds, comparables filter, self-booking flag, JS/Python parity	TC-TRUST-, *TC-FAIR-*
Jobs	Scrape fail-closed, delisting, expiry completion, recompute	TC-BANK-, *TC-JOB-*
Ops	Backup/restore dry runs; migration not applied to live	TC-OPS-*
E2E	Full owner → admin → renter → dispute walkthrough on deployed build	TC-E2E-001

6. Security Test Matrix (rules)

Run each row as: signed-out, renter, owner, unrelated user, suspended user, admin. Expected = allow (A) or deny (D).

Operation	Out	Renter	Owner	Other	Susp.	Admin
Read verified listing	A	A	A	A	A	A
Read pending listing	D	D	A	D	D*	A
Read listingPrivate	D	D	A	D	D	A
Set verificationStatus verified	D	D	D	D	D	A
Create booking on verified rental	D	A	D	A	D	A/D†
Pending → Confirmed	D	D	A	D	D	policy
Write depositReference on Confirmed	D	A	D	D	D	policy
Resolve dispute	D	D	D	D	D	A
Write trustScores / bankProperties	D	D	D	D	D	D (Admin SDK only)
Read `__admins__` notifications	D	D	D	D	D	A
Change own role/status	—	D	D	D	D	n/a

* Suspended owners: confirm intended behaviour in rules. † Admin booking as a normal user follows renter rules. “policy” = decide and record in TH-03.

7. Defect and Gap Management

- Severity: **Critical** (data leak, security bypass, data loss), **High** (core flow broken), **Medium** (workaround exists), **Low** (cosmetic).
- Every defect: ID, title, steps, expected vs actual, environment, commit, screenshot, owner, linked TC.
- Known gaps that are design limitations (not bugs) go to the **Known Gaps** sheet with exit criteria and are disclosed in demo notes.
- A fix is closed only when its linked test passes on the target environment and the evidence label is updated.
- Regression: re-run all High/Critical tests of a module after any change to that module, and the full Security rows after any rules change.

8. Non-functional Checks

Check	Method	Target
Accessibility	axe/Lighthouse + keyboard + NVDA/VoiceOver on key flows	No serious violations; WCAG 2.2 AA contrast; visible focus
Performance	Lighthouse mobile on Home, Browse, Listing detail	LCP < 2.5 s on 4G throttling; no main bundle warning
Resilience	DevTools offline/slow 3G during upload, booking, admin load	Loading fallback, clear error, retry; no blank screen
Compatibility	Chrome, Edge, Firefox, Android Chrome, iOS Safari	Layouts intact at 360/390/768/1280
Privacy	Inspect network responses and page source as signed-out user	No addresses, document URLs, emails of other users
Secrets	`git ls-files | grep -i key`, secret scanning, Actions logs review	No credentials anywhere